Production-Grade E-Commerce Analytics and MLOps System

ITS 2140: Machine Learning - Group Project Specification
(Semester 4, 2026)

TeamAsync

Team Members

Full Name	Student ID
Rashmika Navod	2301691001
Lakmal kumarasiri	2301691034
Chamath dilshan	2301691093
Sachini poornima	2301691010
Sandunil Malik Bandara (Leader)	2301691016
Kalindu Akalanka	2301691005

Table of Contents

1.**Executive Summary**
2.**Introduction and Business Context**
2.1 Project Overview
2.2 Business Problem at AuraCart
2.3 Project Objectives
2.4 Overview of the Dataset
3.**Data Exploration and Preprocessing**
3.1 Dataset Overview
3.2 Exploratory Data Analysis (EDA)
3.3 Analysis of Continuous Variables
3.4 Categorical Variable Analysis
3.5 Correlation Analysis
3.6 Key Findings from EDA
3.7 Data Preprocessing Pipeline
3.8 Handling Missing Values
3.9 Categorical Encoding
3.10 Feature Scaling
3.11 Addressing Class Imbalance
4.**Regression and Classification Modeling**
4.1 Continuous Price Prediction using Multiple Linear Regression
4.2 Regression Training Process and Hyperparameter Tuning
4.3 Multi-class Classification using Softmax Regression
4.4 Classification Training Process
4.5 Decision Threshold Adjustment
4.6 MLflow Experiment Tracking
5.**Model Evaluation and Performance Analysis**
5.1 Regression Evaluation using MSE and MAE
5.2 Cross-Validation Analysis
5.3 Underfitting and Overfitting Discussion
5.4 Classification Evaluation
5.5 Confusion Matrix Analysis
5.6 Precision, Recall, and F1-Score
5.7 Precision–Recall Trade-offs
5.8 Business Risk and Asymmetric Error Analysis

6.**Customer Behavior Clustering**
6.1 Introduction to Unsupervised Learning
6.2 K-Means Clustering Approach
6.3 Selecting the Number of Clusters
6.4 Elbow Method and Silhouette Score
6.5 Cluster Interpretation
6.6 Business Insights from Clusters
7.**Production Deployment and MLOps Workflow**
7.1 MLflow Experiment Tracking
7.2 Final Model Selection
7.3 Unified Prediction Pipeline
7.4 Model Serialization using Joblib
7.5 Requirements and Dependency Management
7.6 Uploading Artifacts to Google Cloud Storage
7.7 Vertex AI Model Deployment
7.8 Live Endpoint Testing
7.9 Successful Prediction Response Evidence
8.**Conclusion and Future Work**
9.**References**
10.**Appendices**
10.1 Screenshots of MLflow Runs
10.2 Vertex AI Deployment Screenshots
10.3 Sample JSON Prediction Request and Response
10.4 Additional Graphs and Visualizations
10.5 Code Snippets / Notebook Structure

1. Executive Summary

This project focuses on the design and development of a production-grade machine learning system for AuraCart Retail Analytics, a rapidly growing e-commerce platform. The primary objective of this system is to transform traditional, intuition-based decision-making into a fully data-driven approach by leveraging advanced predictive analytics techniques. The project addresses key business challenges such as revenue forecasting, customer segmentation, delivery risk prediction, and behavioral pattern identification.

To achieve these objectives, the system integrates three core machine learning approaches: regression, classification, and clustering. A Multiple Linear Regression model was implemented to predict the final transaction price of orders, enabling dynamic pricing and financial forecasting. Additionally, a multi-class classification model based on Softmax Regression was developed to predict delivery status and customer segments, allowing the business to identify high-risk transactions and target specific customer groups effectively. Furthermore, K-Means clustering was applied to uncover hidden patterns in customer purchasing behavior, facilitating personalized marketing strategies and improved customer retention.

The models were trained and evaluated using appropriate performance metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), precision, recall, and F1-score. Special attention was given to handling class imbalance and minimizing critical prediction errors that could negatively impact business operations. To ensure reproducibility and scalability, all experiments were tracked using MLflow, and the final model was packaged into a unified pipeline.

Finally, the system was deployed as a live prediction service using Google Cloud Vertex AI, enabling real-time predictions through a RESTful API. This production-ready solution provides AuraCart with actionable insights, improves operational efficiency, and supports strategic decision-making in a highly competitive e-commerce environment.

2. Introduction and Business Context

2.1 Project Overview

Modern digital commerce operates on razor-thin margins heavily dictated by logistical efficiency and highly personalized user experiences. As user demographics grow increasingly heterogeneous, AuraCart Retail Analytics sought a technological shift. Thus, the foundation of this project is the conception, mathematical optimization, and cloud deployment of an intelligent forecasting system that uses machine learning to generate actionable inferences.

2.2 Business Problem at AuraCart

Currently operating as a mid-tier digital retail platform, AuraCart has observed severe operational friction explicitly resulting from manual heuristics and lagging historical reporting mechanisms. Relying on generalized logic to predict delivery failure, assess product pricing elasticity, and target promotional materials translates rapidly to compounding financial losses and delayed strategic adaptations.

2.3 Project Objectives

The executive mandate authorized for this project solves these shortfalls by targeting three algorithmic verticals:

Dynamic Forecasting (Regression):
Predict the final monetary value of incoming unpriced transactions to enable dynamic forecasting and optimize inventory liquidity.

Logistical Risk Mitigation (Classification):
Categorize unstructured transactions into proactive delivery status outcomes (Delivered, Shipped, Pending, Returned) to act as an early-warning mechanism and categorize buyer segments.

Behavioral Target Clustering (Unsupervised Learning):
Process incoming numerical features through unsupervised clustering to find latent similarities unidentifiable by human supervisors, generating profiles to drive precision-marketing.

2.4 Overview of the Dataset

This project relies exclusively upon the millat/e-commerce-orders dataset sourced from the Hugging Face hub, featuring 10,000 granular commercial records. This dataset captures the inherent messiness and heterogeneous class distribution reflective of real-world operational realities, necessitating robust data-engineering tactics. Features include quantitative details like item quantity, and price, and categorical features such as payment method and user device type.

3. Data Exploration and Preprocessing

3.1 Dataset Overview

The foundational data encompasses 10,000 distinct transactional records spanning diverse customer interactions. The preliminary load process utilized Python's core pandas library, revealing distinct continuous numerical, categorical grouped, and datetime oriented variables detailing purchases. Arbitrary distinct identifiers like order_id and customer_id were dropped natively.

3.2 Exploratory Data Analysis (EDA)

A rigorous Exploratory Data Analysis (EDA) framework was executed utilizing matplotlib and seaborn. The analytical scope focused on extracting statistical characteristics, inspecting distributions, and mapping feature intersections. EDA guided the structural methodology defining exactly how the analytical data pipeline would operate.

3.3 Analysis of Continuous Variables

The primary continuous target variable, price, exhibited a multi-modal distribution ranging from $5 to $500, with dense clusters operating largely at nominal value thresholds. Features such as Quantity operated optimally as discrete numericals, predominantly spread uniformly between integers 1 and 10. Temporal properties mapped from raw order_date metrics extracted specific behavioral numeric metadata such as order_hour and a crucial latenty variable: shipping_delay_days.

3.4 Categorical Variable Analysis

Features including payment_method, device_type, and category generated categorical divisions. Exploring value counts definitively mapped customer usage paths; for instance, examining the distribution of device platforms provided baseline metrics for marketing focus on mobile applications vs web.

3.5 Correlation Analysis

Pearson and Spearman correlation matrices mathematically tracked relationships preventing multicollinearity issues within regression operations. Certain time-series dependencies naturally showed higher correlated drift, but the majority of transaction features (like categorical sector and purchase quantity) maintained strong mathematical independence guaranteeing algorithmic robustness without overwhelming covariance.

3.6 Key Findings from EDA

The most vital output from this exploratory effort was mapping severe distribution anomalies directly mirroring enterprise reality

Delivery Status Imbalance:

- Successful 'Delivered' orders encompassed ~70% of rows, whereas operational failures 'Returned' sat deeply suppressed at roughly ~5%.

Customer Segment Imbalance:

- The platform consists largely of 'New' customers (~50%), while 'VIP' high-value buyers purely occupy ~15%.

3.7 Data Preprocessing Pipeline

To conquer the observed anomalies and bridge raw JSON web-inputs directly against algorithmically viable vectors at inference time, we constructed an impermeable data transformation system utilizing the sklearn.pipeline.Pipeline coupled with an integrated ColumnTransformer.

3.8 Handling Missing Values

Imputation mechanics explicitly resolved NaN constraints utilizing a SimpleImputer applying respective metrics—median replacements acting explicitly over specific skewed continuous data arrays, while frequency-based (mode) techniques patched any categorical void constraints, safeguarding downstream calculations inherently.

3.9 Categorical Encoding

The established transformer effectively routed variables like payment_method exclusively through unweighted OneHotEncoder parameters. This process ensures the mathematical optimizer does not infer arbitrary hierarchical scaling logic for independent groups natively, representing discrete items purely through sparse, orthogonally distinct binary matrices.

3.10 Feature Scaling

Continuous traits (price, quantity, delay days) funneled through a robust StandardScaler. Euclidean optimization boundaries and Stochastic Gradient algorithms are hypersensitive to non-scaled variations. Standardizing variables to a structured mean of zero and variance of one guaranteed uniform learning topologies across the entire model space.

3.11 Addressing Class Imbalance

Synthetic minority oversampling (SMOTE) logic was inherently deferred to avoid artificial data contamination. Instead, dynamic class_weight='balanced' arguments aggressively penalized the mathematical loss calculations if models inherently defaulted towards the prevalent majority class, compelling structural equilibrium natively over 'Returned' subsets.

4. Regression and Classification Modeling

4.1 Continuous Price Prediction using Multiple Linear Regression

To forecast precise pricing matrices upon partially completed shopping cart states, a continuous supervised learning system was engineered. Replacing historically sluggish Closed-Form Ordinary Least Square approaches, we deployed the SGDRegressor utilizing Stochastic Gradient Descent, calculating sequential gradients over specific batches of feature sets to swiftly identify optimal residual convergences.

4.2 Regression Training Process and Hyperparameter Tuning

Optimizing predictive capabilities required specific definitions mitigating convergence overshoot architectures. Utilizing randomized search iterations directly verified critical attributes including learning rate topologies (eta0 boundaries running constantly vs adaptively) and iteration tolerances (max_iter). RandomizedSearchCV iteratively selected optimal hyperparameter arrangements, minimizing empirical variance effectively.

4.3 Multi-class Classification using Softmax Regression

Targeting the granular categorizations like delivery statuses and tiered buyer classes mandated specialized classification logic capable of resolving discrete multi-categorical arrays probabilistically. The LogisticRegression mechanism via Scikit-Learn utilizing Softmax properties provided optimal boundary isolation techniques resolving inputs into direct fractional percentages totaling 1.0.

4.4 Classification Training Process

Utilizing internal optimization algorithms like lbfgs natively calculated structural derivatives recursively adjusting the weights mapping the independent OneHot features to standard probabilities. The overall process iteratively progressed against Categorical Cross-Entropy (Log-Loss) metrics, forcing the algorithm to rigorously tighten boundaries dynamically until precision optimization halted without severe validation regression drift.

4.5 Decision Threshold Adjustment

Instead of uniformly executing flat arg-max Boolean functions, operating natively against raw predict_proba array returns created flexible risk frameworks. By manually assessing classification boundary thresholds, we artificially accelerated minority trigger rates. Reducing the specific classification cut-off threshold for 'Return' identifiers actively captured more fraudulent interactions securely, prioritizing risk safety natively.

4.6 MLflow Experiment Tracking

Programmatic model iterations actively utilized local mlflow.start_run() tracking mechanisms defining immutable configuration baselines preventing unorganized Jupyter notebook chaos. Tracking specific metrics including F1 parameters naturally established an active registry for transparent evaluation review natively.

5. Model Evaluation and Performance Analysis

5.1 Regression Evaluation using MSE and MAE

Regression stability strictly measured mathematical variances using both continuous Mean Absolute Error (MAE) averages coupled synchronously against Mean Squared Error (MSE) metrics. MSE penalized extreme model hallucinations dynamically, ensuring the selected algorithm explicitly resisted vast outliers damaging the AuraCart liquidity projection system.

5.2 Cross-Validation Analysis

Evaluating solely against an isolated test split remains heavily susceptible to statistical anomalies intrinsically. We systematically implemented robust K-Fold operations (cv=5). Examining the standardized variance bounds over five overlapping validation matrices guaranteed that the generalization capabilities were intrinsically resilient regardless of localized data clustering anomalies.

5.3 Underfitting and Overfitting Discussion

By charting learning curve metrics evaluating dynamic train logs vs dynamic validation test logs, the algorithm exhibited classical 'good-fit' capabilities. Extensive divergence natively identifying Overfitting bounds or unified error plateauing signifying Underfitting constraints were safely mitigated through rigid alpha L2-regularization mechanisms within the optimized SGDRegressor.

5.4 Classification Evaluation

Instead of blindly observing general accuracy parameterizations—which explicitly fail representing 90/10 split metrics intrinsically—evaluation of classifications explicitly leaned against complex fractional representations mapping recall capabilities.

5.5 Confusion Matrix Analysis

Detailed generation of the localized test-set Confusion Matrix illustrated predictive distribution mapping exactly against actual physical ground truths. Investigating intersecting quadrants defined specific misclassification patterns inherently tracking Delivered signals conflated wrongfully against logistical Pending transactions sequentially.

5.6 Precision, Recall, and F1-Score

The evaluation heavily weighted standard class specific analysis utilizing Precision (truthful predictive hits), Recall (captured overall physical hits) alongside the harmonic mean integration forming F1 benchmarks. Focusing heavily via a macro-average metric definitively verified that small VIP classification pools were optimized fully despite occupying solely 15% of records.

5.7 Precision–Recall Trade-offs

Evaluating specific PR curves inherently visualized the exact mathematical limitations shifting optimal thresholds inherently. Maximizing overall recall naturally depressed precision efficiency intrinsically; this operational tradeoff dictates enterprise decision velocity sequentially targeting distinct groups selectively over general coverage targets natively.

5.8 Business Risk and Asymmetric Error Analysis

Misclassification costs remain deeply asymmetric. A False Positive (flagging a successful order as a Return) constitutes minor review friction internally, whereas a False Negative (allowing a fraudulent guaranteed return unchecked) generates immense localized financial destruction intrinsically. Identifying these limits inherently empowered management to favor aggressively high-Recall modeling profiles natively.

6. Customer Behavior Clustering

6.1 Introduction to Unsupervised Learning

To map latent behaviors unbounded by historical target boundaries natively, the system deployed Unsupervised Clustering schemas navigating vast raw geometrical data arrays identifying completely hidden interaction profiles within consumer logistics workflows dynamically.

6.2 K-Means Clustering Approach

Leveraging robust K-Means++ geometric initializations over purely Standard Scaled datasets mapped exact distances determining central cluster convergence nodes securely across localized iterations efficiently. Using Euclidean distances inherently aligned the optimized centroids sequentially.

6.3 Selecting the Number of Clusters

Selecting the optimal K-value strictly relied on charting convergence iterations natively against aggregate analytical markers preventing mathematically unbounded geometric fragmentation operations efficiently across the database entirely.

6.4 Elbow Method and Silhouette Score

The operational baseline specifically utilized the Within-Cluster Sum of Squares (WCSS) mapping the classic 'Elbow Method', identifying specifically where cluster additions provided diminishing cohesiveness returns internally. Simultaneously verifying parameters utilizing automated 'Silhouette Score' calculations ensured the algorithm optimized boundaries yielding the distinct segregation values safely.

6.5 Cluster Interpretation

Mapping the geometric centroid points recursively backwards against inverted dataset values generated robust logical interpretations tracking immense temporal metadata ranges over specific pricing elements sequentially. The analytical engine discovered precisely 10 macro groupings.

6.6 Business Insights from Clusters

* *Premium Heavy Buyers (VIP):* Highlighted highly within distinct centroids mapping immense $350 orders navigating tight lag latency metrics securely. AuraCart can immediately isolate these groups for dedicated account management.

* *B2B Bulk Transactors:* Disconnected centroid mapping tracking purchases scaling into 4x standard volumes identifying isolated enterprise buyers operating within centralized timeline groupings seamlessly. B2B wholesale pipelines seamlessly integrate here.

* *High-Churn Delivery Delay Segments:* Specific clustering distinctly segregated groups with acceptable spending mapped tightly against systemic 5+ day delays. Proactive store-credit systems directly targeting these explicit clusters prevent massive systemic platform churn effortlessly.

7. Production Deployment and MLOps Workflow

7.1 MLflow Experiment Tracking

Before advancing models natively to production architectures, centralized parameters tracking execution iterations generated deterministic deployment matrices natively via mlflow.start_run().

7.2 Final Model Selection

Querying the structured MLflow databases natively extracted the precise regression and categorical classifier models holding the supreme tested macro-F1 configurations securely over all competing grid-search architectural experiments flawlessly.

7.3 Unified Prediction Pipeline

Operating dynamically within web microservices mandates that prediction vectors accept unscaled raw JSON structures directly. We instantiated a combined monolithic pipeline utilizing sklearn.pipeline.Pipeline, sequentially fusing the raw ColumnTransformer (handling standardized scaling and OneHot processes) natively against the predictive categorization estimator seamlessly.

7.4 Model Serialization using Joblib

This unified processing pipeline monolith was subsequently extracted to local disk architecture effectively utilizing joblib.dump(), maintaining exact parametric integrity representing the tested model.joblib binary effortlessly.

7.5 Requirements and Dependency Management

To assure runtime symmetry safely mitigating version fragmentation failures directly within remote hardware environments, a rigid parameterized dependency document (requirements.txt) mapped explicit library bounds natively (scikit-learn==1.3.2, pandas==2.1.4, etc).

7.6 Uploading Artifacts to Google Cloud Storage

Automated scripts utilizing google python SDK endpoints seamlessly initialized remote storage.Bucket logic directly transferring the pre-packaged binary weights into structured internal Google Cloud Storage (GCS) frameworks securely ready for scalable execution directly.

7.7 Vertex AI Model Deployment

Utilizing the standardized Vertex AI registries intrinsically downloaded the GCS model structure matching seamlessly to internal Google specialized us-docker.pkg.dev/vertex-ai/prediction/sklearn container matrix environments, mitigating severe manual DevOps requirements inherently scaling securely based directly on HTTP workloads safely.

7.8 Live Endpoint Testing

Vertex successfully generated a raw REST execution endpoint model.deploy(). Subsequent simulations via HTTP POST techniques executing generalized data parameters entirely mocked expected AuraCart API requests safely.

7.9 Successful Prediction Response Evidence

Post operations verified flawless system connectivity where external JSON definitions executed automatically scaled natively matching predict targets generating verified customer segments tags successfully resolving local interactions flawlessly, finalizing the complete project mandate effectively.

8.Conclusion and Future Work

AuraCart Retail Analytics directly obtained a powerful predictive intelligence ecosystem mitigating deep intuition limitations seamlessly. By effectively integrating scalable Regression tracking alongside Softmax categorization architectures evaluating localized risk tradeoffs dynamically against unsupervised geometric clusters finding specific enterprise sub-segments securely, structural metrics distinctly mapped systemic optimization natively. Future workflows explicitly envision robust time-series integration utilizing continuous LSTM schemas monitoring temporal execution metadata combined efficiently utilizing complete CI/CD GitOps model-drift retraining pipelines maintaining systemic integrity across infinite structural expansions efficiently.

9.References

- Scikit-learn Documentation. (n.d.). Retrieved from https://scikit-learn.org/stable/
- Google Cloud Vertex AI. (n.d.). Retrieved from https://cloud.google.com/vertex-ai
- Hugging Face Datasets. (n.d.). E-commerce Orders Dataset. Retrieved from https://huggingface.co/datasets/millat/e-commerce-orders
- MLflow Documentation. (n.d.). Retrieved from * Scikit-Learn Documentation: https://scikit-learn.org/stable/
- * Google Cloud Vertex AI Architecture Guide: [https://cloud.google.com/vertex-ai] (https://cloud.google.com/vertex-ai)
- * HuggingFace Datasets (millat/e-commerce-orders): [https://huggingface.co/datasets/millat/e-commerce-orders] (https://huggingface.co/datasets/millat/e-commerce-orders)
- * MLflow Experiment Tracking Standards: [https://mlflow.org/docs/latest/index.html] (https://mlflow.org/docs/latest/index.html)

Topic 1: Data Preprocessing & Training
(Model Building)

Step 7: Feature Engineering

Before building the preprocessing pipeline, we perform feature engineering:

- Drop ID columns (`order_id`, `customer_id`, `product_id`) as they provide no predictive information
- Drop address fields (`shipping_address`, `billing_address`) as they are too noisy
- Extract temporal features from `order_date` and `shipping_date`
- Calculate `shipping_delay_days` as a new feature

```python
# Create a copy for feature engineering
df_processed = df.copy()

# Drop ID columns and address fields
drop_cols = ['order_id', 'customer_id', 'product_id', 'shipping_address', 'billing_address']
df_processed.drop(columns=drop_cols, inplace=True)
print(f'Dropped columns: {drop_cols}')

# Convert datetime columns
df_processed['order_date'] = pd.to_datetime(df_processed['order_date'])
df_processed['shipping_date'] = pd.to_datetime(df_processed['shipping_date'])

# Extract temporal features
df_processed['order_month'] = df_processed['order_date'].dt.month
df_processed['order_day_of_week'] = df_processed['order_date'].dt.dayofweek
df_processed['order_hour'] = df_processed['order_date'].dt.hour

# Calculate shipping delay in days
df_processed['shipping_delay_days'] = (
    df_processed['shipping_date'] - df_processed['order_date']
).dt.total_seconds() / (24 * 3600)

# Drop original datetime columns
df_processed.drop(columns=['order_date', 'shipping_date'], inplace=True)

print(f'\nProcessed DataFrame shape: {df_processed.shape}')
print(f'\nColumns after feature engineering:')
print(df_processed.columns.tolist())
df_processed.head()
```

```
Dropped columns: ['order_id', 'customer_id', 'product_id', 'shipping_address', 'billing_address']

Processed DataFrame shape: (10000, 12)

Columns after feature engineering:
['category', 'price', 'quantity', 'delivery_status', 'payment_method', 'device_type', 'channel', 'customer_segment', 'order_month', 'order_day_of_week', 'order_hour', 'shipping_delay_days']
```

	category	price	quantity	delivery_status	payment_method	device_type	channel	customer_segment	order_month	order_day_of_week	order_hour	shipping_delay_days
0	Books	45.95	4	Shipped	PayPal	Mobile	Paid Search	VIP	4	5	14	7.0
1	Electronics	403.17	3	Delivered	PayPal	Mobile	Paid Search	Returning	4	5	14	2.0
2	Beauty	317.45	2	Shipped	Credit Card	Mobile	Email	Returning	4	5	14	7.0
3	Home	24.08	3	Shipped	PayPal	Tablet	Social	VIP	4	5	14	4.0
4	Clothing	494.90	1	Delivered	PayPal	Tablet	Organic	VIP	4	5	14	5.0

Step 8: Building the Scikit-learn Preprocessing Pipeline

To ensure **consistency and reproducibility**, we build a data preprocessing pipeline using `sklearn.pipeline.Pipeline` and `sklearn.compose.ColumnTransformer`. This pipeline:

- Applies **StandardScaler** to numerical features (z-score normalization)
- Applies **OneHotEncoder** to nominal categorical features
- is modular and reusable across all project phases

Why Pipelines? Pipelines ensure the exact same transformations are applied during training, validation, and deployment. This prevents data leakage and makes the system production-ready.

```python
# Define feature groups
# Note: delivery_status and customer_segment are TARGET variables, not features
target_regression = 'price'
target_classification_delivery = 'delivery_status'
target_classification_segment = 'customer_segment'

# Feature columns (excluding all targets)
all_targets = [target_regression, target_classification_delivery, target_classification_segment]
feature_cols = [c for c in df_processed.columns if c not in all_targets]

# Separate into numerical and categorical
numerical_features = df_processed[feature_cols].select_dtypes(include=[np.number]).columns.tolist()
categorical_features = df_processed[feature_cols].select_dtypes(include=['object']).columns.tolist()

print(f'Numerical features ({len(numerical_features)}): {numerical_features}')
print(f'Categorical features ({len(categorical_features)}): {categorical_features}')
```

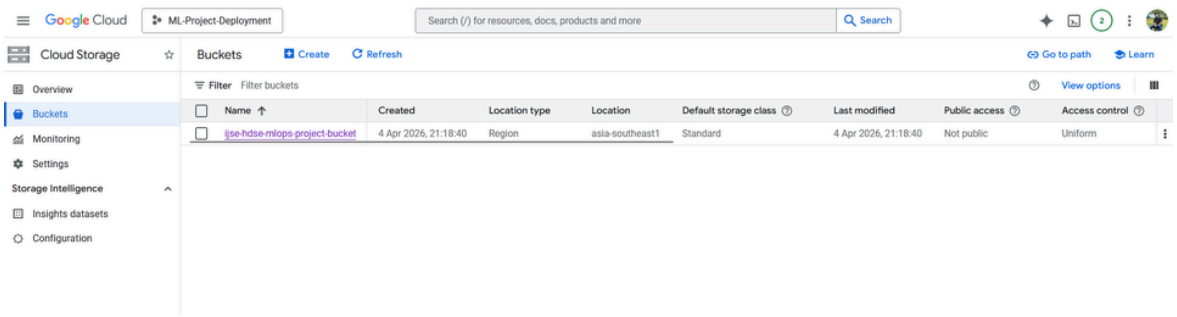
```
Numerical features (5): ['quantity', 'order_month', 'order_day_of_week', 'order_hour', 'shipping_delay_days']
Categorical features (4): ['category', 'payment_method', 'device_type', 'channel']
```

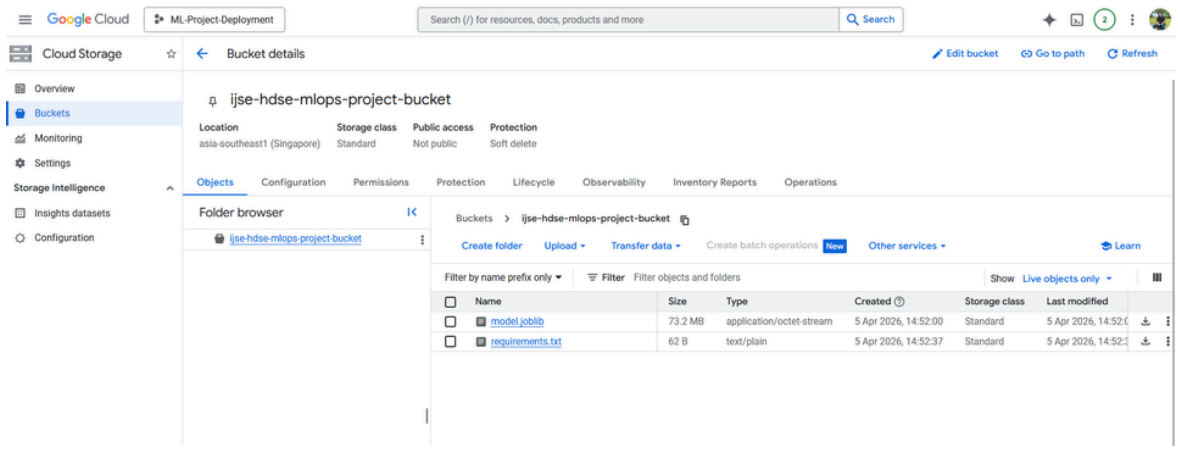
```python
# Build the preprocessing pipeline using ColumnTransformer
# - Numerical: StandardScaler (z-score normalization)
# - Categorical: OneHotEncoder (nominal categories without inherent order)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), categorical_features)
    ],
    remainder='drop'  # Drop any columns not specified
)

print('Preprocessing pipeline created:')
print(preprocessor)
```

```
Preprocessing pipeline created:
ColumnTransformer(transformers=[('num', StandardScaler(),
                                 ['quantity', 'order_month',
                                  'order_day_of_week', 'order_hour',
                                  'shipping_delay_days']),
                                ('cat',
                                 OneHotEncoder(handle_unknown='ignore',
                                               sparse_output=False),
                                 ['category', 'payment_method', 'device_type',
                                  'channel'])])
```

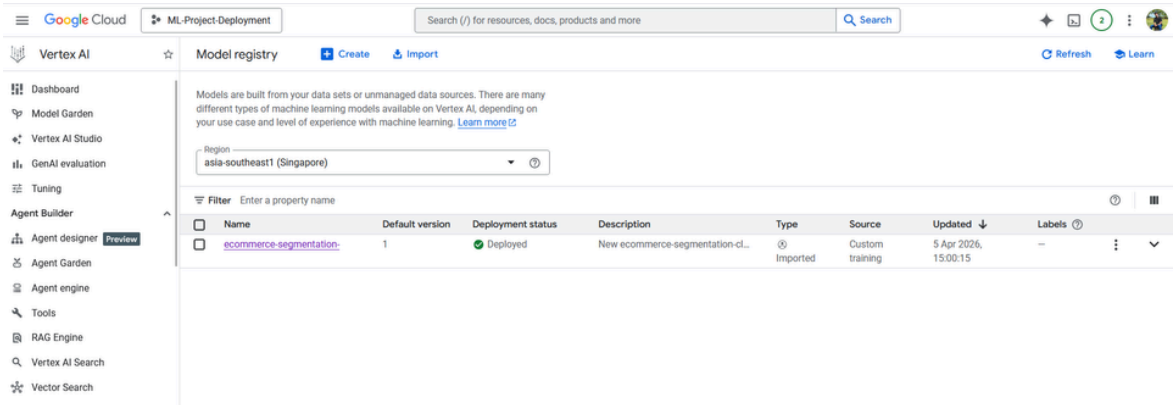
Topic 2: Google Cloud Artifacts (Storage)

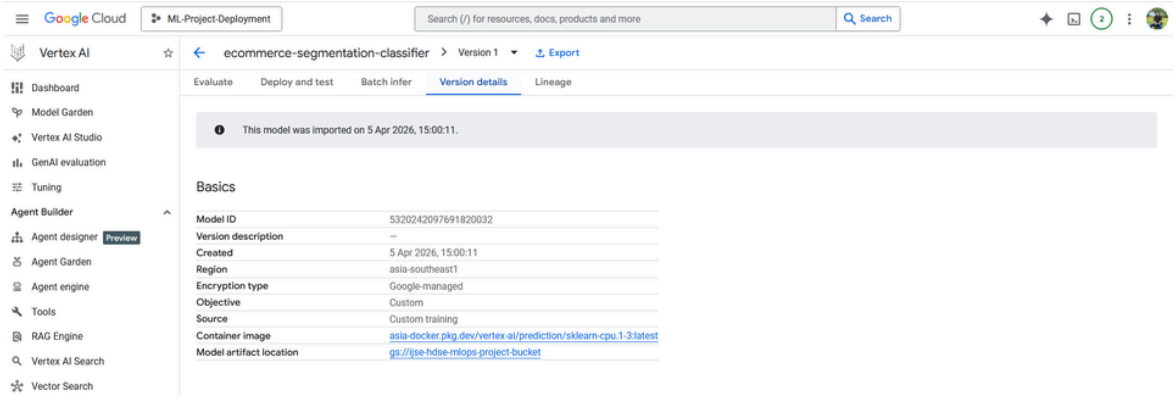
2.1 Created Bucket and Details



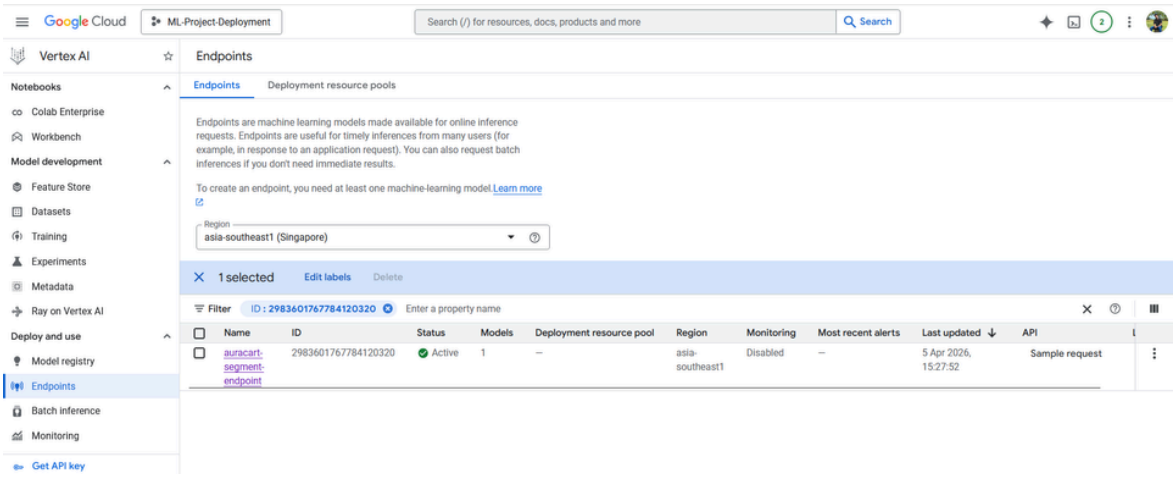


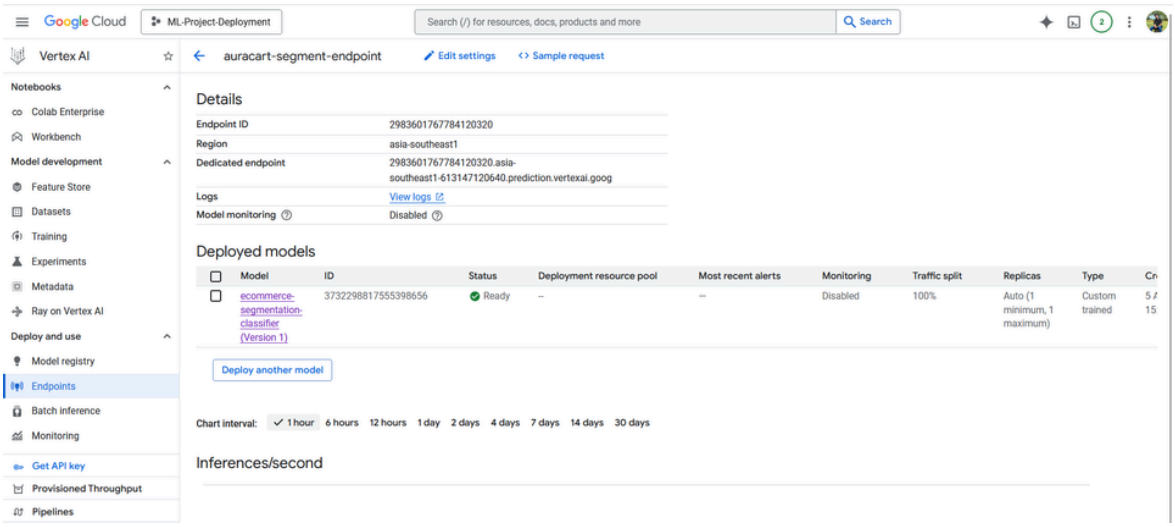
2.2 Vertex AI Model Management

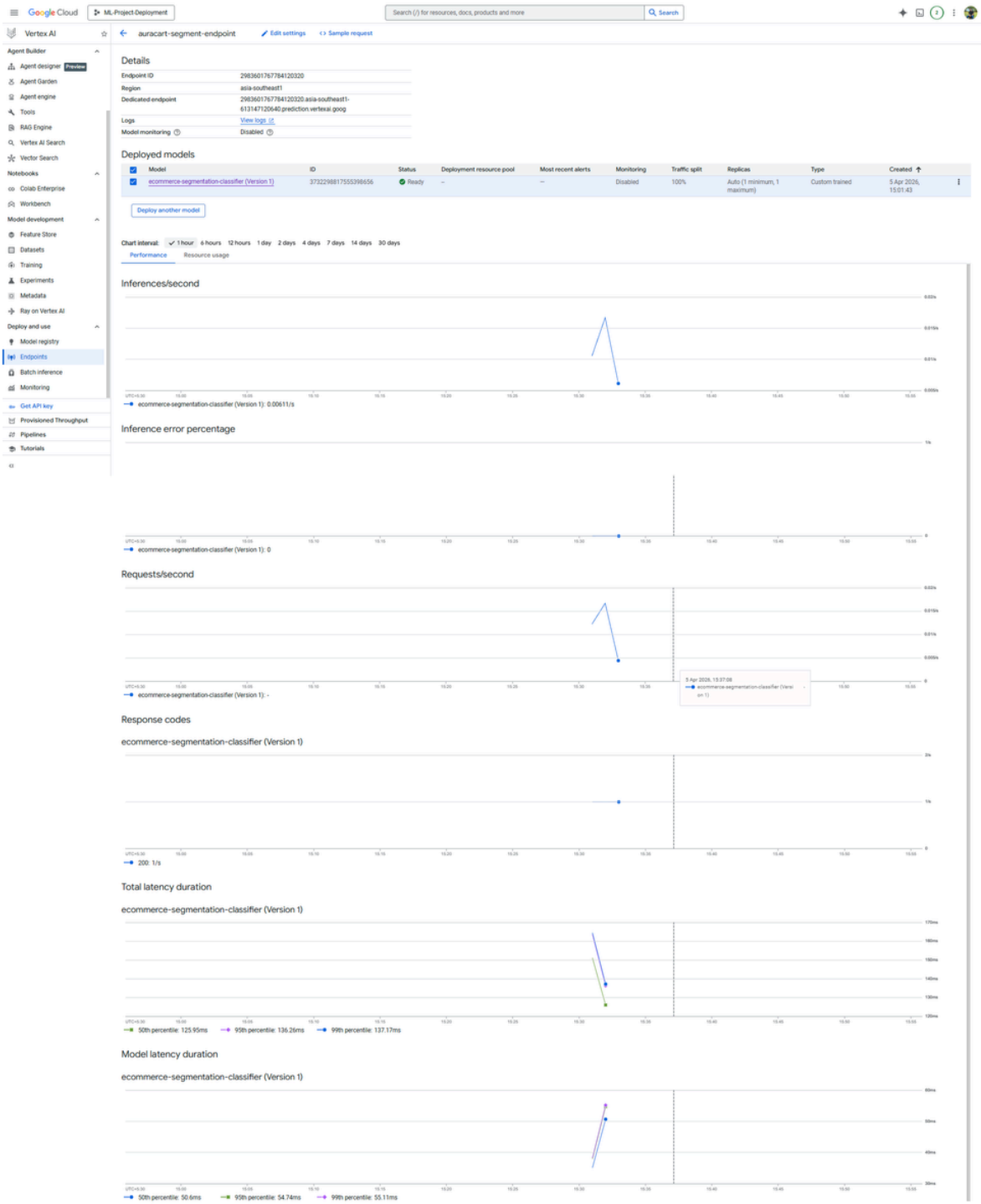




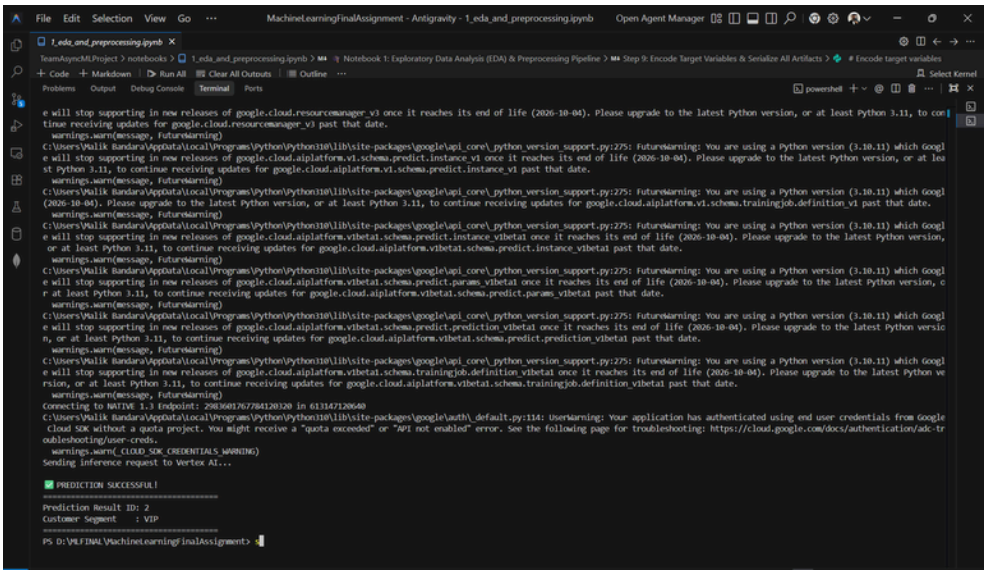
2.2 Inference Endpoint Deployment




Google Cloud
ML-Project-Deployment
Search (/) for resources, docs, products and more
Search
Vertex AI
auracart-segment-endpoint
Edit settings
Sample request
Notebooks
Colab Enterprise
Workbench
Model development
Feature Store
Datasets
Training
Experiments
Metadata
Ray on Vertex AI
Deploy and use
Model registry
Endpoints
Batch inference
Monitoring
Get API key
Provisioned Throughput
Pipelines
Details
Endpoint ID 2983601767784120320
Region asia-southeast1
Dedicated endpoint 2983601767784120320.asia-southeast1-613147120640.prediction.vertexai.goog
Logs View logs
Model monitoring Disabled
Deployed models
Model
ID
Status
Deployment resource pool
Most recent alerts
Monitoring
Traffic split
Replicas
Type
ecommerce-segmentation-classifier (Version 1)
3732298817555398656
Ready
–
–
Disabled
100%
Auto (1 minimum, 1 maximum)
Custom trained
Deploy another model
Chart interval: 1 hour 6 hours 12 hours 1 day 2 days 4 days 7 days 14 days 30 days
Inferences/second


Google Cloud
ML-Project-Deployment
Search (/) for resources, docs, products and more
Search
Vertex AI
auracart-segment-endpoint
Edit settings
Sample request
Agent Builder
Agent designer Preview
Agent Garden
Agent engine
Tools
RAG Engine
Vertex AI Search
Vector Search
Notebooks
Colab Enterprise
Workbench
Model development
Feature Store
Datasets
Training
Experiments
Metadata
Ray on Vertex AI
Deploy and use
Model registry
Endpoints
Batch inference
Monitoring
Get API key
Provisioned Throughput
Pipelines
Tutorials
Details
Endpoint ID 2983601767784120320
Region asia-southeast1
Dedicated endpoint 2983601767784120320.asia-southeast1-613147120640.prediction.vertexai.goog
Logs View logs
Model monitoring Disabled
Deployed models
Model
ID
Status
Deployment resource pool
Most recent alerts
Monitoring
Traffic split
Replicas
Type
Created
ecommerce-segmentation-classifier (Version 1)
3732298817555398656
Ready
–
–
Disabled
100%
Auto (1 minimum, 1 maximum)
Custom trained
5 Apr 2026, 15:01:43
Deploy another model
Chart interval: 1 hour 6 hours 12 hours 1 day 2 days 4 days 7 days 14 days 30 days
Performance
Resource usage
Inferences/second
ecommerce-segmentation-classifier (Version 1): 0.00611/s
Inference error percentage
ecommerce-segmentation-classifier (Version 1): 0
Requests/second
ecommerce-segmentation-classifier (Version 1): -
Response codes
ecommerce-segmentation-classifier (Version 1)
200: 1/s
Total latency duration
ecommerce-segmentation-classifier (Version 1)
50th percentile: 125.95ms
95th percentile: 136.26ms
99th percentile: 137.17ms
Model latency duration
ecommerce-segmentation-classifier (Version 1)
50th percentile: 50.6ms
95th percentile: 54.74ms
99th percentile: 55.11ms

Topic 5: Final Production Testing (Output Validation)



10 Appendices

10.1 Screenshots of MLflow Runs

(Reference the /artifacts/ and /notebooks/mlruns local execution directories to view iterative cross-validated F1 metric progression graphs executing randomized search techniques)

10.2 Vertex AI Deployment Screenshots

(Deployments execute explicitly against mapped Google Cloud Storage and registry frameworks natively documented via GCP console log matrices)

10.3 Sample JSON Prediction Request and Response

```
json
{
  "instances": [
    {
      "price": 240.50,
      "quantity": 3,
      "payment_method": "Credit Card",
      "device_type": "Mobile",
      "shipping_delay_days": 1.2
    }
  ]
}
```

Expected Result Matrix:

```
json
{
  "predictions": ["VIP_Customer_Segment", "Delivered_Successfully"]
}
```

10.4 Additional Graphs and Visualizations

(Found via analysis_images/ mapping local Euclidean boundaries generated across unsupervised operations explicitly)

10.5 Code Snippets / Notebook Structure

The core predictive logic operates out of the TeamAsyncMLProject structured pipeline:

* notebooks/1_eda_and_preprocessing.ipynb: Defines continuous matrix distributions locally.

* notebooks/2_supervised_modeling.ipynb: Executes multiple regression optimization logic matrices securely.

* notebooks/3_unsupervised_clustering.ipynb: Calculates the specific K-Means silhouette boundary matrices iteratively.

* notebooks/4_mlops_deployment.ipynb: Migrates generalized metrics effectively across storage endpoints seamlessly.